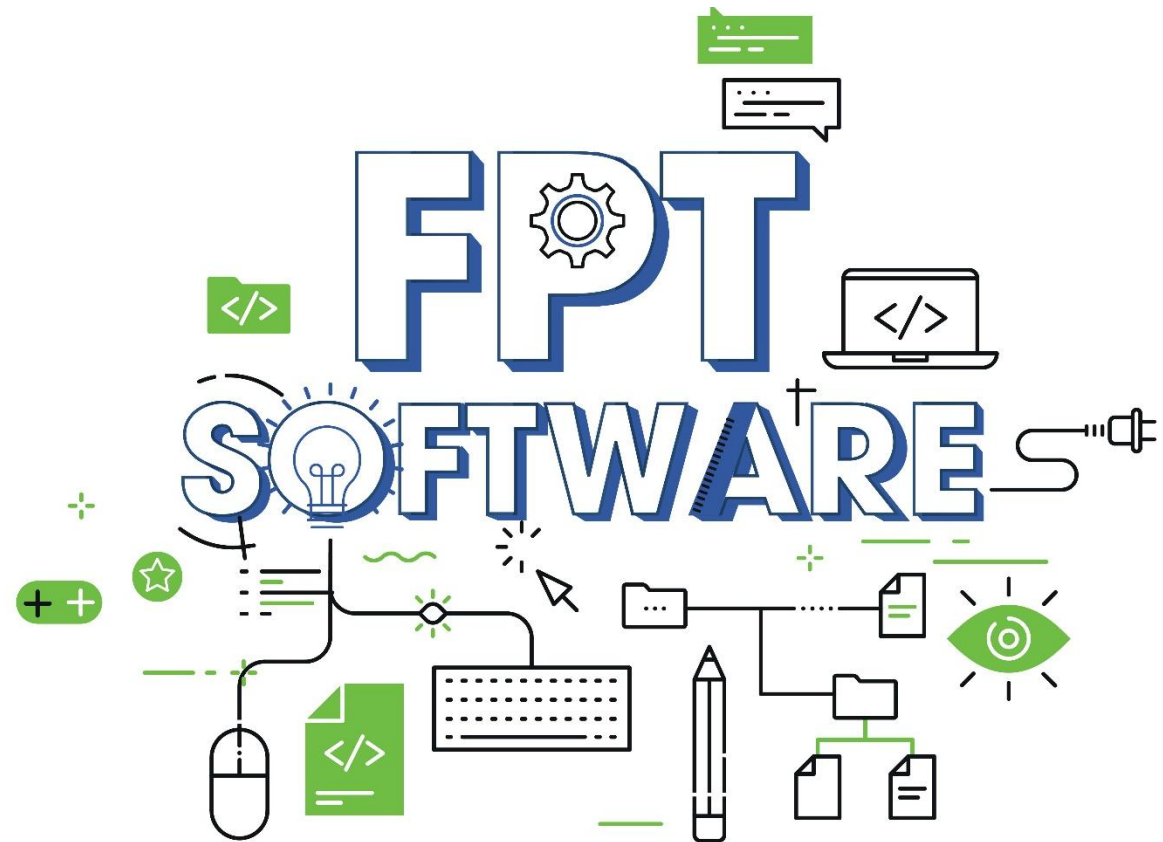


Basic Android

ViewGroup



1.

ANDROID UI LAYOUT

2.

ANDROID UI CONTROLS

3.

ANDROID EVENT HANDLING

4.

Improve layout performance

5.

Create custom view components



ANDROID UI LAYOUT

Android UI Layout

The **ViewGroup** is a subclass of **View** and provides invisible container that hold other

Views or other ViewGroups and define their layout properties.

There are many different layouts which are subclasses of ViewGroup

There are many ways to create UI for the Android Application

- ✓ XML
- ✓ Java
- ✓ “OpenGL”
- ✓ ...

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical" >

    <Button
        android:id="@+id/button2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:onClick="buttonOnClick2"
        android:text="Button2" />
</LinearLayout>
```

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.test);
}
```

Android UI Layout – View group

Linear Layout	• LinearLayout is a view group that aligns all children in a single direction, vertically or horizontally
Relative Layout	• RelativeLayout is a view group that displays child views in relative positions.
Table Layout	• TableLayout is a view that groups views into rows and columns.
Absolute Layout	• AbsoluteLayout enables you to specify the exact location of its children. (This class was deprecated in API level 3.)
Frame Layout	• The FrameLayout is a placeholder on screen that you can use to display a single view.
List View	• ListView is a view group that displays a list of scrollable items
Grid View	• GridView is a ViewGroup that displays items in a two-dimensional, scrollable grid.
ScrollView	• The ScrollView class can be used to contain one View that might be too big to fit on one screen

Android UI Layout

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent" >

    <TextView
        android:layout_alignParentTop="true"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/hello_world" />

</RelativeLayout>
```

Android UI Layout - Attribute

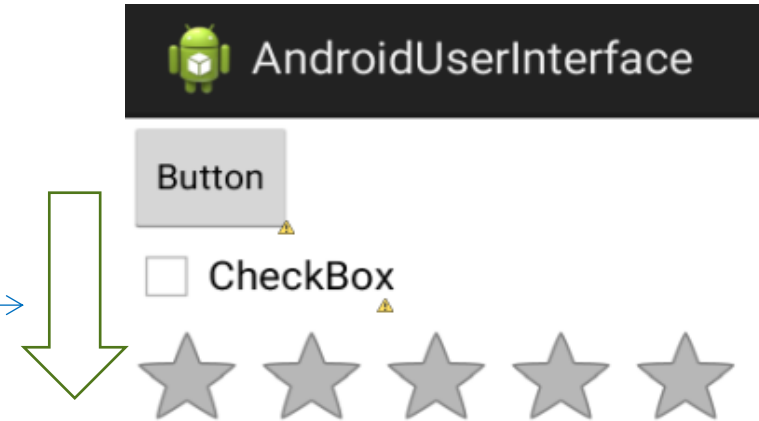
Attribute	Description
android:id	This is the ID which uniquely identifies the view.
android:layout_width	This is the width of the layout.
android:layout_height	This is the height of the layout
android:layout_marginTop	This is the extra space on the top side of the layout.
android:layout_marginBottom	This is the extra space on the bottom side of the layout.
android:layout_marginLeft	This is the extra space on the left side of the layout.
android:layout_marginRight	This is the extra space on the right side of the layout.

Android UI Layout - Attribute

Attribute	Description
android:layout_gravity	This specifies how child Views are positioned.
android:layout_weight	This specifies how much of the extra space in the layout should be allocated to the View.
android:paddingLeft	This is the left padding filled for the layout.
android:paddingRight	This is the right padding filled for the layout.
android:paddingTop	This is the top padding filled for the layout.
android:paddingBottom	This is the bottom padding filled for the layout.

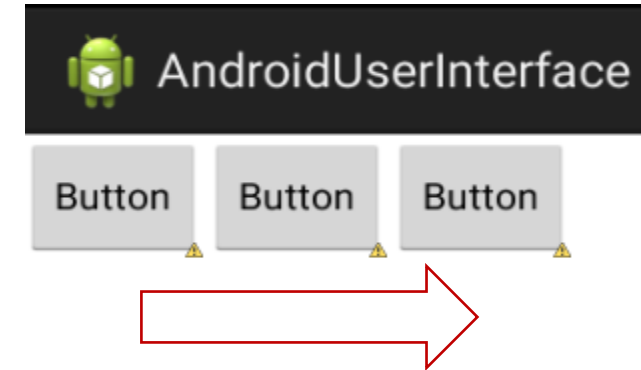
Android UI Layout – LinearLayout vertical

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:orientation="vertical" >
  <Button
    android:id="@+id/button3"
    style="?android:attr/buttonStyleSmall"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Button" />
  <CheckBox
    android:id="@+id/checkbox1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="CheckBox" />
  .....
</LinearLayout>
```



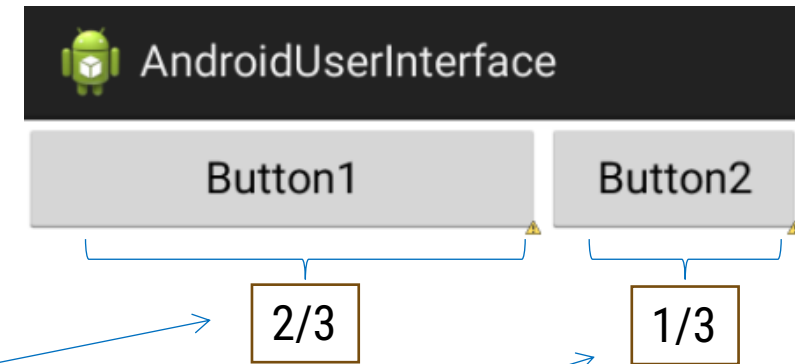
Android UI Layout – LinearLayout horizontal

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:orientation="horizontal" >
  <Button
    android:id="@+id/button3"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Button" />
  <Button
    android:id="@+id/button4"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Button" />
  .....
</LinearLayout>
```



Android UI Layout – LinearLayout weight

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/an
droid"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:weightSum="3"
    android:orientation="horizontal" >
    <Button
        android:layout_weight="2"
        android:id="@+id/button3"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="Button1" />
    <Button
        android:layout_weight="1"
        android:id="@+id/button4"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="Button2" />
</LinearLayout>
```

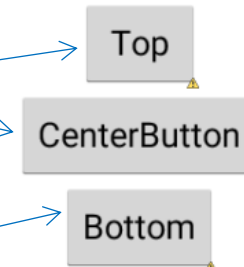


Android UI Layout – RelativeLayout



AndroidUserInterface

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent" >
    <Button
        android:id="@+id/btnCenter"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerInParent="true"
        android:text="CenterButton" />
    <Button
        android:id="@+id/btnTop"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_above="@+id/btnCenter"
        android:layout_centerInParent="true"
        android:text="Top" />
    <Button
        android:id="@+id/btnBottom"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@+id/btnCenter"
        android:layout_centerInParent="true"
        android:text="Bottom" />
</RelativeLayout>
```



ANDROID UI CONTROLS

Android UI Controls

UI Controls	Description
TextView	This control is used to display text to the user
EditText	EditText is a predefined subclass of TextView that includes rich editing capabilities.
AutoCompleteTextView	The AutoCompleteTextView is a view that is similar to EditText, except that it shows a list of completion suggestions automatically while the user is typing.
Button/ImageButton	A push-button that can be pressed, or clicked, by the user to perform an action.
CheckBox	An on/off switch that can be toggled by the user.

Android UI Controls

UI Controls	Description
ToggleButton	An on/off button with a light indicator.
RadioGroup/RadioButton	The RadioButton has two states: either checked or unchecked.
TimePicker/DatePicker	The view enables users to select a time of the day/date of the day
ProgressBar	The ProgressBar view provides visual feedback about some ongoing tasks, such as when you are performing a task in the background.
Spinner	A drop-down list that allows users to select one value from a set.

ANDROID EVENT HANDLING

Android Event Handling

Event Handler	Event Listener & Description
onClick()	OnClickListener() This is called when the user either clicks or touches or focuses upon any widget like button, text, image etc. You will use onClick() event handler to handle such event.
onLongClick()	OnLongClickListener() This is called when the user either clicks or touches or focuses upon any widget like button, text, image etc. for one or more seconds. You will use onLongClick() event handler to handle such event.
onFocusChange()	OnFocusChangeListener() This is called when the widget loses its focus ie. user goes away from the view item. You will use onFocusChange() event handler to handle such event.
onKey()	OnKeyListener() This is called when the user is focused on the item and presses or releases a hardware key on the device. You will use onKey() event handler to handle such event.

Android Event Handling

Event Handler	Event Listener & Description
onTouch()	OnTouchListener() This is called when the user presses the key, releases the key, or any movement gesture on the screen. You will use onTouch() event handler to handle such event.
onMenuItemClick()	OnMenuItemClickListener() This is called when the user selects a menu item. You will use onMenuItemClick() event handler to handle such event.

```
tvDelayTime.setOnClickListener(new OnClickListener() {  
    @Override  
    public void onClick(View arg0) {  
        // Todo  
    }  
});
```

Improve layout performance

Improve layout performance

- Layouts are a key part of Android applications that directly affect the user experience. If implemented poorly, your layout can make your app memory-intensive with slow UIs. The Android SDK includes tools to help identify problems in your layout performance. With this documentation, you can implement smooth scrolling interfaces with a minimal memory footprint.
 - ✓ Optimize layout hierarchies
 - ✓ Reuse layouts with `<include>`
 - ✓ Load views on demand

Optimize layout hierarchies

- It is a common misconception that using the basic layout structures leads to the most efficient layouts. However, each widget and layout you add to your application requires initialization, layout, and drawing. For example, using nested instances of **LinearLayout** can lead to an excessively deep view hierarchy. Furthermore, nesting several instances of **LinearLayout** that use the *layout_weight* parameter can be especially expensive as each child needs to be measured twice. This is particularly important when the layout is inflated repeatedly, such as when used in a **ListView** or **GridView**.

Reuse layouts with `<include>`

- Although Android offers a variety of widgets to provide small, reusable, interactive elements, you might also need to reuse larger components that require a special layout. To efficiently reuse complete layouts, use the `<include>` and `<merge>` tags to embed one layout inside another.
- This lets you create complex layouts—such as a yes or no button panel or a custom progress bar with description text. And it means that you can extract any elements of your application that are common across multiple layouts, manage them separately, and include them in each layout. While you can create individual UI components by writing a custom View, you can do it more easily by reusing a layout file.

Load views on demand

- Sometimes your layout requires complex views that are rarely used. Whether they are item details, progress indicators, or undo messages, you can reduce memory usage and speed up rendering by loading the views only when they're needed.
- You can defer loading resources when you have complex views that your app needs in the future by defining a ViewStub for complex and rarely used views.

Create custom view components

Create a view class (1)

- A well-designed custom view is like any other well-designed class. It encapsulates a specific set of functionality with a simple interface, uses CPU and memory efficiently, and so on. In addition to being a well-designed class, a custom view must do the following:
 - ✓ Conform to Android standards.
 - ✓ Provide custom styleable attributes that work with Android XML layouts.
 - ✓ Send accessibility events.
 - ✓ Be compatible with multiple Android platforms.
- The Android framework provides a set of base classes and XML tags to help you create a view that meets all of these requirements. This lesson discusses how to use the Android framework to create the core functionality of a view class.

Create a view class (2) - Subclass a view

- All the view classes defined in the Android framework extend View. Your custom view can also extend View directly, or you can save time by extending one of the existing view subclasses, such as Button.
- To allow Android Studio to interact with your view, at a minimum you must provide a constructor that takes a Context and an AttributeSet object as parameters. This constructor allows the layout editor to create and edit an instance of your view.

```
class PieChart(context: Context, attrs: AttributeSet) : View(context, attrs)
```

Create a view class (3) - Define custom attributes

- To add a built-in View to your user interface, specify it in an XML element and control its appearance and behavior with element attributes. You can also add and style custom views using XML. To enable this behavior in your custom view, do the following:
 - ✓ Define custom attributes for your view in a <declare-styleable> resource element.
 - ✓ Specify values for the attributes in your XML layout.
 - ✓ Retrieve attribute values at runtime.
 - ✓ Apply the retrieved attribute values to your view.

Create a view class (4) - Define custom attributes

- To define custom attributes, add <declare-styleable> resources to your project. It's customary to put these resources into a res/values/attrs.xml file. Here's an example of a attrs.xml file:

```
<resources>
  <declare-styleable name="PieChart">
    <attr name="showText" format="boolean" />
    <attr name="labelPosition" format="enum">
      <enum name="left" value="0"/>
      <enum name="right" value="1"/>
    </attr>
  </declare-styleable>
</resources>
```

Create a view class (5) - Define custom attributes

- Once you define custom attributes, you can use them in layout XML files just like built-in attributes. The only difference is that your custom attributes belong to a different namespace. Instead of belonging to the **http://schemas.android.com/apk/res/android** namespace, they belong to **http://schemas.android.com/apk/res/[your package name]**. For example, here's how to use the attributes defined for PieChart:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:custom="http://schemas.android.com/apk/res-auto">
    <com.example.customviews.charting.PieChart
        custom:showText="true"
        custom:labelPosition="left" />
</LinearLayout>
```

Create a view class (6) - Apply custom attributes

- When a view is created from an XML layout, all the attributes in the XML tag are read from the resource bundle and passed into the view's constructor as an **AttributeSet**. Although it's possible to read values from the **AttributeSet** directly, doing so has some disadvantages:
 - ✓ Resource references within attribute values are not resolved.
 - ✓ Styles are not applied.

```
init {  
    context.theme.obtainStyledAttributes(  
        attrs,  
        R.styleable.PieChart,  
        0, 0).apply {  
  
        try {  
            mShowText = getBoolean(R.styleable.PieChart_showText, false)  
            textPos = getInteger(R.styleable.PieChart_labelPosition, 0)  
        } finally {  
            recycle()  
        }  
    }  
}
```

Custom Drawing

✓ The most important part of a custom view is its appearance. Custom drawing can be easy or complex according to your application's needs. This lesson covers some of the most common operations.

- Override onDraw()
- Create Drawing Objects
- Handle Layout Events
- Draw!
- Apply graphics effects

THANK YOU!

